DEVNET.CSU · PARTICIPANT GUIDE

Pokerbots: Heads-Up Tournament

Everything your team needs to build, test, and submit a poker bot

Version 3 · November 2026

Your team writes a program that plays poker for you. Bots play each other one-on-one for thousands of hands, and the bot that beats the most opponents wins. You only write the decision-making part. Dealing, rules, timing, and scoring are all handled for you.

At a glance

Game	Heads-up (one-on-one) No-Limit Texas Hold'em
Your job	Write `get_action()`, one Python method that picks fold, check, call, or raise
Teams	1–4 people, open to any CSU student who joins DevNet.CSU
Language	Python 3.12, with the standard library, eval7, and NumPy
A matchup	Two 3,000-hand games against one opponent, with the same cards dealt in both and the seats swapped
Chips	Every hand starts fresh at 400 chips per bot, with blinds of 1 and 2. Your bankroll starts at 10,000 chips, just for show.
Thinking time	180 seconds per bot for each 3,000-hand game
Format	Every bot plays every other bot once
Winner	Most matchups won; total chips breaks ties
Timeline	Three weeks from kickoff to the final

In this guide

1 **Glossary** · poker and tournament terms

2 **Rules of the game** · how a hand plays out, betting, hand rankings

3 **How the tournament works** · where your bot runs and what it can see

4 **Building your bot** · setup, the code you write, testing

5 **Scoring and awards** · duplicate matches, standings, the four awards

6 **Schedule and submitting** · deadlines, GitHub, the final

7 **Rules of conduct** · what's allowed, penalties, the bug bounty

A **Appendix** · what the engine actually sends your bot, and credits

Questions? Ask Kacper Mazur, DevNet.CSU president, at any club meeting.

1 Glossary

Terms in `code font` are names you'll see in the starter code. Cards are written as a rank (2–9, T, J, Q, K, A) followed by a suit (c, d, h, s), so `Ts` is the ten of spades.

Poker terms

Term	Meaning
Heads-up	Only two players at the table.
No-limit	You may bet any amount, up to all the chips you have in the hand.
Hole cards	Your two private cards. Only you (and the engine) can see them.
Board	Up to five shared cards dealt face up in the middle. Also called community cards.
Street	A betting round. Preflop (no board cards), flop (3), turn (4th card), river (5th card).
Button	The dealer position. In heads-up, the button posts the small blind, acts first preflop, and acts last on every later street.
Blinds	Forced bets before any cards, so every hand has something to win: the small blind (1 chip) and big blind (2 chips).
Pot	All chips bet so far in the hand. The winner of the hand takes it.
Stack	The chips you have left to bet in this hand.
Check	Pass without betting. Only allowed when nobody has bet on this street.
Bet / raise	Put chips in / increase the current bet. Both are `RaiseAction` in the code.
Call	Match the current bet to stay in the hand.
Fold	Give up the hand, losing whatever you've already put in the pot.
All-in	Bet every chip you have left.
Min-raise	The smallest legal raise: at least the size of the previous bet or raise, and never less than the big blind.
Showdown	After the last betting round, both hands are revealed and the best five-card hand wins.
Bluff	Betting with a weak hand to make the opponent fold a better one.
Equity	Your chance of winning the pot if the hand went to showdown from here. Bots often estimate it by simulating thousands of random run-outs (Monte Carlo).
Pot odds	How much you must call compared with what you'd win. Calling 10 into a pot that will be 40 pays off if you win more than 25% of the time.

Tournament terms

Term	Meaning
Engine	The program that shuffles, deals, enforces the rules, and asks each bot for its moves. It's the only program that knows every card.
Bot	Your team's program. It decides one action at a time when the engine asks.
Skeleton	Starter code that handles all communication with the engine. You fill in `get_action()` and leave the `skeleton/` folder unchanged.
Hand (round)	One deal, from the blinds to awarding the pot. The code calls a hand a *round*.
Game	3,000 hands between the same two bots. Your bot starts at the beginning of a game and stops at the end.
Matchup	Two games against the same opponent. See *duplicate match*.
Duplicate match	Two games with identical cards and the seats swapped, so card luck mostly cancels out.
Round-robin	A format where every bot plays every other bot once.
Bankroll	`game_state.bankroll`: your net chips won or lost so far in this game.
Pip	`round_state.pips`: chips a bot has put in on the *current street*. Resets to 0 on each new street.
Game clock	Your bot's total thinking time for a game. It only ticks while the engine is waiting for your bot's answer.
House bot	A sample opponent in the starter kit, for testing. House bots don't compete for awards.
Official machine	The computer the final runs on, with each bot in its own locked-down container.
Game log	`gamelog.txt`: a hand-by-hand record of a game, including both bots' cards.

2 Rules of the game

The game is standard heads-up No-Limit Texas Hold'em. The only difference from casino poker is that chip stacks reset every hand. Every hand is independent, and no bot can ever be knocked out.

2.1 A hand, step by step

1. **Blinds.** The button posts the small blind (1 chip); the other bot posts the big blind (2 chips).
2. **Hole cards.** Each bot is dealt two private cards.
3. **Preflop betting.** The button acts first.
4. **Flop.** Three board cards are dealt face up, then a betting round. From here on, the big blind acts first.
5. **Turn.** A fourth board card, then a betting round.
6. **River.** A fifth board card, then the final betting round.
7. **Showdown.** If neither bot folded, each makes its best five-card hand from its two hole cards plus the five board cards. The better hand wins the pot; identical hands split it.

A hand ends the moment someone folds. The other bot wins the pot without revealing its cards. The button switches every hand, so your bot is the small blind in exactly half the hands.

2.2 Who acts first

Street	Board cards	Acts first	
Preflop	0	Button (small blind)	Heads-up special case
Flop	3	Big blind	
Turn	4	Big blind	
River	5	Big blind	

At a full table, the player after the big blind acts first preflop. Heads-up flips that. The engine handles it, so all you need to know is who's first.

2.3 Betting rules

- **Checking** is only allowed when no one has bet on this street. Facing a bet, your options are fold, call, or raise.
- **Minimum raise:** increase the bet by at least the size of the previous bet or raise, and by at least the big blind. Preflop, the small blind facing 2 chips can raise to 4 at minimum. If one bot bets 10 and the other raises to 30, the next re-raise must go to at least 50.
- **Maximum raise:** all-in, capped at what the opponent can actually match.
- **Amounts are "raise to," not "raise by."** `RaiseAction(30)` means your total for this street becomes 30. This is the most common beginner bug.
- **After an all-in is called,** no more betting happens. The remaining cards are dealt and both bots check through to showdown.
- **No busting out.** Both stacks reset to 400 at the start of every hand. Your score is the running total of chips won and lost.

2.4 Hand rankings, strongest to weakest

Hand	Example	Description
Straight flush	`9s 8s 7s 6s 5s`	Five in a row, same suit. A-K-Q-J-T suited is a royal flush.
Four of a kind	`Qc Qd Qh Qs 3d`	Four cards of one rank
Full house	`Jc Jd Js 4h 4s`	Three of a kind plus a pair
Flush	`Ah Jh 8h 5h 2h`	Five cards of the same suit
Straight	`Td 9c 8h 7s 6d`	Five ranks in a row. The ace can also be low: A-2-3-4-5.
Three of a kind	`7c 7d 7h Ks 2d`	Three cards of one rank
Two pair	`Ac Ad 9s 9h 4c`	Two different pairs
One pair	`Kd Kh Js 8c 3h`	Two cards of one rank
High card	`Ac Qd 9h 6s 3c`	None of the above; highest cards decide

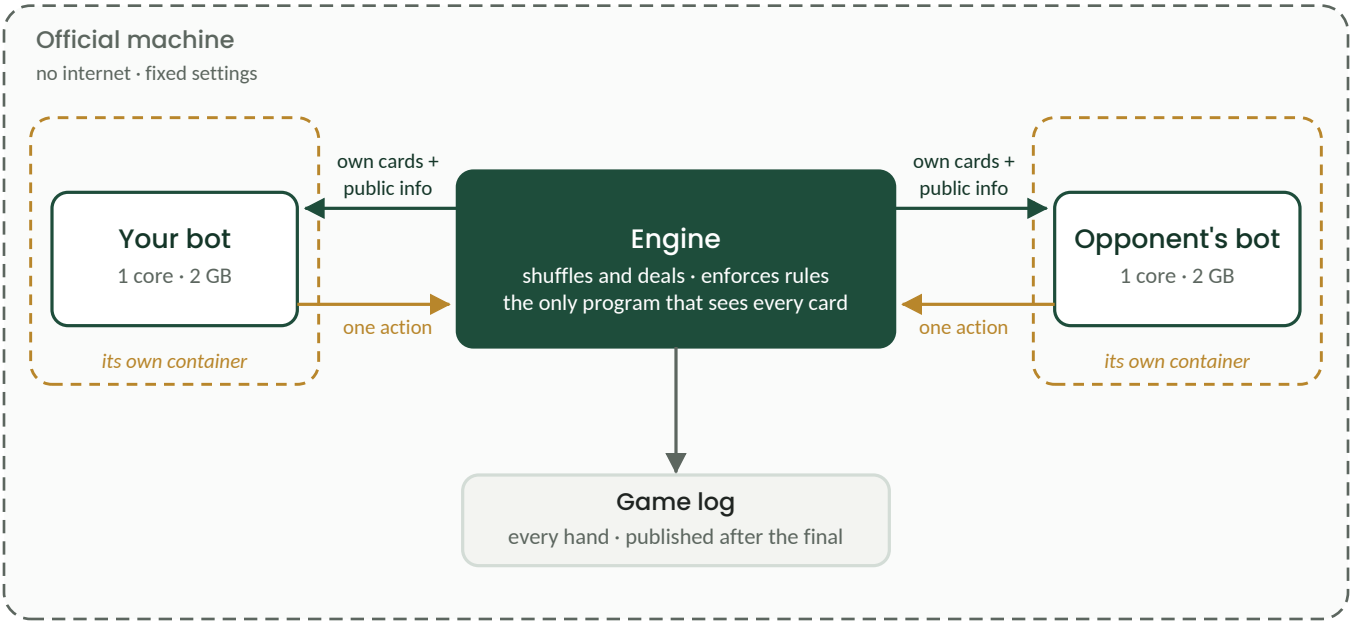
Ties within a category are broken by the cards' ranks (a pair of kings beats a pair of queens, then the remaining "kicker" cards decide). You never need to code this yourself: `eval7.evaluate()` scores any 5–7 card hand.

2.5 When something goes wrong

If your bot...	The engine...
Returns an illegal action (e.g., checks while facing a bet, or raises outside the allowed range)	Checks for you if that's legal, otherwise folds, and notes it in the game log
Sends a garbled reply	Does the same
Crashes, disconnects, or runs out of clock	Checks or folds for your bot for the rest of that game, which almost always means losing chips every hand
Takes more than 10 seconds on a single decision	Counts it as out of time, even with clock left, and checks or folds for it for the rest of that game
Takes more than 10 seconds to start	Gives up on it for that game

3 How the tournament works

Bots never run inside the engine. Each bot is a separate program in its own locked-down container, and the engine only sends it what that bot is allowed to know: its own cards, the board, and the betting so far. The engine's code is public, and that's fine. Knowing how it works doesn't reveal a single hidden card.



3.1 What your bot gets on the official machine

- **1 CPU core and 2 GB of memory.** Your laptop is probably faster, so leave some margin.
- **A 180-second game clock** for each 3,000-hand game. Time only counts while the engine is waiting for your bot, and no single decision may take more than 10 seconds.
- **No internet or network access.** Nothing your bot does can reach outside the container.
- **Read-only access to your own bot folder.** Your bot can read its files but can't change them. Anything it writes to `/tmp` disappears when the game ends.
- **A fresh start every game.** The official machine always runs `python3 player.py` and ignores `commands.json`, so keep `player.py` as your entry point.

3.2 Taking notes on your opponent

Your bot can learn about its opponent during a game. It keeps running for all 3,000 hands, so anything you record in `handle_round_over()` stays in memory and can shape later decisions: how often the opponent folds to a raise, how big it bets with strong hands, whether its showdown hands match its betting. That's 3,000 hands of evidence per game.

Nothing carries over from one game to the next. That's deliberate: the second game of a matchup replays the first game's cards, so a bot that saved its notes would know its opponent's cards. Each game starts clean, for everyone.

3.3 How the final runs

1. Every team's bot plays every other team's bot once.
2. Each matchup is two 3,000-hand games dealt from a secret seed. The second game replays the same cards with the seats swapped (section 5.2 explains why).
3. The chips from both games are added up. Whoever is ahead wins the matchup.
4. The whole final runs the evening before the final meeting. Results stay secret until the reveal.

4 Building your bot

4.1 What's in the starter kit

File or folder	What it's for
`engine.py`, `config.py`	The engine, so you can run full games on your own laptop. The config is preset to the official settings.
`python_skeleton/`	A working (and very bad) bot that always checks or calls. Copy this folder; it becomes your bot.
House bots	A few sample opponents of increasing strength to test against.
`check_bot.py`	The self-check. Run it before every submission (section 4.7).
This guide	

4.2 Setting up

1. Install Python 3.12, the official version. Use 3.12, not 3.13: eval7 only publishes Windows and macOS installers up to Python 3.12.
2. Clone the starter kit, then install the libraries: `pip install -r requirements.txt`
3. Copy `python_skeleton/` to a folder next to the starter kit, for example `../my_bot`. That folder becomes your submission repo (section 6.3).
4. In `config.py`, set `PLAYER_1_PATH` to your bot's folder (for example `'../my_bot'`) and `PLAYER_2_PATH` to a house bot, such as `'./house_bots/rule_book'`.
5. Run `python3 engine.py`. A 3,000-hand game between simple bots takes a few seconds.

If `python3` isn't found (common on Windows), use `python` instead, and change `python3` to `python` in your bot's `commands.json` for local testing.

4.3 What you write

Everything happens in `player.py`, in a class with four methods:

Method	When it's called	Use it for
`get_action(...)`	Every time it's your turn	**Required.** Return one action.
`handle_new_round(...)`	Start of every hand	Optional. Reset per-hand notes.
`handle_round_over(...)`	End of every hand, with the result and the opponent's cards if they were shown	Optional. Track your opponent's tendencies.
`__init__()`	Once, when the game starts	Optional. Load data files or set up counters.

4.4 What your bot can see

Field	Meaning
`round_state.legal_actions()`	The set of actions allowed right now
`round_state.raise_bounds()`	(min, max) legal "raise to" amounts
`round_state.street`	0 = preflop, 3 = flop, 4 = turn, 5 = river (equals the number of board cards)
`round_state.hands[active]`	Your two cards, e.g. ['As', 'Kd']
`round_state.deck[:street]`	The board cards dealt so far
`round_state.pips[active]`	Chips you've put in on this street. Use [1 - active] for the opponent.
`round_state.stacks[active]`	Chips you have left this hand. Use [1 - active] for the opponent.
`active`	Your seat this hand: 0 = button/small blind, 1 = big blind. It flips every hand.
`game_state.bankroll`	Your net chips so far this game
`game_state.game_clock`	Seconds of thinking time you have left
`game_state.round_num`	Hand number, from 1 to 3,000
`terminal_state.deltas[active]`	In handle_round_over: chips you won or lost this hand
`terminal_state.previous_state.hands[1 - active]`	In handle_round_over: the opponent's cards if they were shown, otherwise []

Your method returns one of `FoldAction()`, `CheckAction()`, `CallAction()`, or `RaiseAction(amount)`. Bets and raises are both `RaiseAction`, and the amount is the total you'll have in on this street.

4.5 A first bot

```
def get_action(self, game_state, round_state, active):
    legal = round_state.legal_actions()
    street = round_state.street             # 0 preflop, 3 flop, 4 turn, 5 river
    my_cards = round_state.hands[active]    # e.g. ['As', 'Kd']
    board = round_state.deck[:street]       # board cards so far
    continue_cost = round_state.pips[1 - active] - round_state.pips[active]

    ranks = [card[0] for card in my_cards]
    strong = ranks[0] == ranks[1] or 'A' in ranks or 'K' in ranks

    if strong and RaiseAction in legal:
        min_raise, max_raise = round_state.raise_bounds()
        return RaiseAction(min_raise)       # raise TO the smallest legal amount
    if CheckAction in legal:
        return CheckAction()
    if continue_cost <= 4:                  # cheap to stay in
        return CallAction()
    return FoldAction()
```

Raises with any pair, ace, or king; otherwise checks when it's free and calls small bets. In a 1,000-hand test game, it made no illegal actions and beat the always-check-or-call skeleton by 534 chips. A start, not a strategy.

Common mistakes

- Returning an action that isn't in `legal_actions()`. The engine checks or folds for you, which is rarely what you wanted.
- Treating `RaiseAction(amount)` as "add this much." It's the new total for the street.
- Slow code. 180 seconds across 3,000 hands is about 60 ms per hand. A Monte Carlo loop with 100,000 samples per decision will burn the clock within a few hands. Check `game_state.game_clock` and scale your effort to it. No single decision may take more than 10 seconds.
- Forgetting that your seat flips every hand. Always index with `active`, never a hard-coded 0 or 1.
- Trying to save files. Your folder is read-only on the official machine, so keep notes in memory.
- Printing on every decision. Your output goes to a log capped at 512 KB per game.

4.6 Ideas for getting stronger

Level	Idea
Getting started	Rules based on your hole cards: play pairs and high cards, fold junk, and check whether calling is worth the pot odds.
Intermediate	Monte Carlo equity: deal out the unknown cards thousands of times with eval7 to estimate your chance of winning, and size bets from it.
Advanced	Opponent modeling: track how this opponent plays and exploit it. Or precompute a strategy offline (for example with CFR) and ship it as a data file.

4.7 Testing

- **Play full games locally.** `gamelog.txt` shows every hand, including both bots' cards. Anything your bot prints goes to `A.txt` or `B.txt`, named after the players in `config.py`.

- **Run the self-check before every submission:** `python3 check_bot.py ../my_bot`. It plays 100 hands against a house bot under the official settings (even if you've edited `config.py`) and reports crashes, illegal actions, and how much of your clock you used.
- **Experiment freely.** Change `NUM_ROUNDS` or the clock in `config.py` while testing. The official machine always uses the official settings.
- **Test against every house bot**, not just the weakest one.

5 Scoring and awards

5.1 Round-robin

Every bot plays one matchup against every other bot.

Teams	2	3	4	5
Matchups	1	3	6	10

5.2 Duplicate matches

Poker is noisy: over a few hundred hands, a weak bot holding great cards can beat a strong one. Duplicate matches remove most of that luck. Each matchup is two games that use the same 3,000 deals, with the seats swapped:

	Seat A	Seat B	Cards
Game 1	Your bot	Opponent	Hands 1–3,000 from a secret seed
Game 2	Opponent	Your bot	The same hands 1–3,000

If your bot is dealt pocket aces on hand 37 of game 1, your opponent gets those same aces, in the same seat, on hand 37 of game 2. Both bots get the same cards overall, so what's left is mostly the difference in how they played them.

5.3 Standings

- **Winning a matchup:** your chips from game 1 plus game 2. If the total is positive, you won. Every hand is zero-sum, so your opponent's total is the exact negative.
- **Ranking:** most matchups won.
- **Tiebreak:** total net chips across all your matchups.
- **Bankroll:** each team's 10,000-chip buy-in plus its net chips. It's for show and can even go negative: nobody can run out or be eliminated.

Example standings with four made-up teams:

Rank	Team	vs Full House	vs Bit Flush	vs River Rats	vs Null Pointers	Won	Net chips	Bankroll
1	Full House	—	+1,240	−310	+2,050	2	+2,980	12,980
2	Bit Flush	−1,240	—	+480	+1,600	2	+840	10,840
3	River Rats	+310	−480	—	+900	2	+730	10,730
4	Null Pointers	−2,050	−1,600	−900	—	0	−4,550	5,450

Three teams won two matchups each, so net chips breaks the tie. The ranking rewards beating everyone, not crushing one weak opponent.

5.4 Awards

Award	How it's decided
Champion	First place in the standings. Comes with a prize, announced at kickoff.
Best Bluff	The biggest pot won by making the opponent fold while holding the weaker hand: less than a 50% chance to win if the hand had been played out from there. Found in the final's game logs.
Most Creative Strategy	Awarded at the final, after every team presents its bot.
Bug Bounty	For privately reporting a real problem with the engine or the setup before the submission deadline (section 7.3). Recognition only, so it never changes the standings.

6 Schedule and submitting

6.1 Timeline

The whole competition runs three weeks. Exact dates are announced at kickoff.

When	What happens	Due
Week 1	Kickoff meeting. Starter kit and this guide released.	End of week: team registration and library requests
Week 2	Build and test.	
Week 3	Submission deadline two days before the final, then a 24-hour crash-fix window. The final is at a club meeting at the end of the week.	Deadline: your final submission and any bug reports

> **There are no practice rounds**
>
> The final is the first time your bot plays on the official machine. Run the self-check, test against every house bot, and submit early: a bot that crashes on hand 1 folds for the rest of the game.

6.2 Registering your team

- By the end of week 1, send Kacper your team name and members. Solo teams are welcome.
- No team yet? Say so at kickoff and you'll be matched with one.
- Want a library beyond the standard library, eval7, and NumPy? Request it by the end of week 1. It's approved if it installs from PyPI and needs no network while your bot runs.

6.3 Submitting on GitHub

1. Your repo is your bot folder, not a copy of the whole starter kit. `player.py` and the unchanged `skeleton/` folder go at the top level, with any data files your bot loads (up to 100 MB in total).
2. Make the repo private on GitHub and add `kacperm37` as a collaborator.
3. Add a README with your team name, members, and credit for any outside code, sources, or AI tools you used.
4. Your submission is the latest commit on `main` at the deadline. Push as often as you like before then; commits after the deadline don't count.

Your latest submission is tested on the official machine, and you'll hear if it fails. If your final submission crashes there, you get 24 hours to push one commit that fixes the crash. It has to be only a fix: strategy changes aren't allowed in the crash-fix window.

6.4 The final

- The results are revealed matchup by matchup, followed by the biggest pot and the Best Bluff hand on screen.
- Every team gives a 5-minute talk on how its bot works.
- The awards are presented.

> **Beat the Champion (maybe)**
> If the bots are working and people are up for it, we'll finish with Beat the Champion: volunteers play heads-up against the winning bot, live. It's optional, just for fun, and only happens if everything goes smoothly.

6.5 After the final

- Results become official 48 hours after they're announced. Raise any problems within that window.
- If a bug turns up, the affected matchups are rerun with the same cards. Everything else stands.
- The full game logs and deck seeds are published, so anyone can check the results.
- Your code is published only if your team agrees.

7 Rules of conduct

The short version: build the best bot you can, and win at the table, not around it.

7.1 What's allowed

Allowed	Not allowed
✓ AI coding assistants and open-source poker code, credited in your README	✗ Sharing code with another team, or entering more than one bot per team
✓ Studying your opponent during a game and adapting to it	✗ Coordinating with another team: signals between bots, or losing on purpose to help someone
✓ Any strategy that tries to win, including bluffing and aggressive betting	✗ Interfering with the engine, the official machine, or another bot
✓ Precomputed data files, up to 100 MB in total	✗ Trying to reach the internet, or anything outside your own bot folder
	✗ Thinking outside your turn: background threads or processes that keep computing while your opponent thinks
	✗ Using a bug in the engine or setup instead of reporting it

7.2 Penalties

- **Honest bugs** (crashes, timeouts, illegal moves) get no penalty beyond what the engine already does to that game.
- **Breaking a rule in 7.1** means disqualification from the final.
- Kacper makes the rulings, and every disqualification is reviewed by a second person before it's final.

7.3 The bug bounty

Found a way to break the engine or the setup? Report it privately to Kacper before the submission deadline. You'll earn the Bug Bounty award, and the hole gets fixed for everyone. Using it in the final instead counts as breaking the rules.

A Appendix

A.1 What the engine actually sends your bot

You never need to read or write this: the skeleton's `runner.py` turns these messages into the Python objects your code uses. But if you're curious, messages are short lines of text sent over a local connection. This is a real transcript of the first hand of a test game (with a 30-second clock) in which both bots just checked and called.

Direction	Message	Meaning
Engine → A	`T30.000 P0 HTs,7h`	A has 30 s on its clock, is seat 0 (button, small blind), and holds Ts 7h
A → Engine	`C`	Call (completes the big blind)
Engine → B	`T30.000 P1 H6s,4s C`	B is seat 1 (big blind), holds 6s 4s, and A called
B → Engine	`K`	Check
Engine → B	`T30.000 K B2c,4d,Qs`	B's check went through; the flop is 2c 4d Qs; B acts first now
B → Engine	`K`	Check
Engine → A	`T30.000 C K B2c,4d,Qs K`	Everything since A last spoke: its call, B's check, the flop, B's check
A → Engine	`K`	Check
…		Turn and river: both bots check
Engine → A	`T29.999 K O6s,4s D-2`	A's river check; showdown shows B had 6s 4s (a spade flush); A lost 2 chips
A → Engine	`K`	Acknowledges the end of the hand
Engine → both	`Q`	Game over (sent after the last hand)

Codes: `T` clock in seconds · `P` your seat (0 = button) · `H` your cards · `B` board · `F` fold · `C` call · `K` check · `R##` raise to ## · `O` opponent's cards at showdown · `D` your chip change for the hand · `Q` quit.

A.2 Credits and further reading

- The engine and skeleton come from MIT Pokerbots' open-source reference engine (`github.com/mitpokerbots/engine`), with small changes for this tournament.
- Hand evaluation uses eval7 (`github.com/julianandrews/pyeval7`).
- MIT Pokerbots' past engines and course materials are a good place to find strategy ideas. Credit anything you use in your README.